

```
├── models.py
|
├── routes/
|   └── trips.py
.
```

Afficher plus ▾

Voici une version **complète et propre** de ton projet `travel_api/` avec :

- **Flask Blueprint**
- **SQLAlchemy séparé**
- **Service Layer (`trip_service.py`)**
- **Schemas**
- **Validators**
- **Utils**
- **Application Factory**

Structure finale

`travel_api/`



```
|
├── run.py
├── config.py
├──
└── app/
    ├── __init__.py
    ├── extensions.py
    ├── models.py
    ├── schemas.py
    ├── validators.py
    ├── utils.py
    ├──
    ├── services/
    │   ├── __init__.py
    │   └── trip_service.py
    ├──
    └── routes/
        ├── __init__.py
        └── trips.py
```

1) config.py

</> Python



▶ Exécuter

```
class Config:

    SQLALCHEMY_DATABASE_URI = (
        "sqlite:///travel.db"
    )

    SQLALCHEMY_TRACK_MODIFICATIONS = False
```

2) run.py

Point de démarrage.

</> Python



▶ Exécuter

```
from app import create_app

app = create_app()

if __name__ == "__main__":

    app.run(
        host="127.0.0.1",
        port=5000,
        debug=True
    )
```

3) app/extensions.py

Gestion des extensions Flask.

</> Python



▶ Exécuter

```
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass
```

```
db = SQLAlchemy(  
    model_class=Base  
)
```

4) app/__init__.py

Création de l'application.

</> Python



▶ Exécuter

```
from flask import Flask  
from flask_cors import CORS  
  
from config import Config  
  
from .extensions import db  
  
def create_app():  
  
    app = Flask(__name__)  
  
    app.config.from_object(  
        Config  
    )  
  
    CORS(app)  
  
    db.init_app(app)  
  
    from .routes.trips import trips_bp  
  
    app.register_blueprint(  
        trips_bp  
    )  
  
    with app.app_context():  
  
        from .models import Trip  
  
        db.create_all()
```

```
return app
```

5) app/models.py

Modèle SQLAlchemy.

</> Python



▶ Exécuter

```
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column

from .extensions import db


class Trip(db.Model):

    __tablename__ = "trips"

    id: Mapped[int] = mapped_column(
        primary_key=True
    )

    title: Mapped[str] = mapped_column(
        String(100)
    )

    destination: Mapped[str] = mapped_column(
        String(100)
    )

    country: Mapped[str] = mapped_column(
        String(2)
    )

    description: Mapped[str] = mapped_column(
        String(255)
    )

    price: Mapped[float]
```

```
duration: Mapped[int]

travel_type: Mapped[str] = mapped_column(
    String(20)
)

image_filename: Mapped[str] = mapped_column(
    String(150)
)

is_available: Mapped[bool] = mapped_column(
    default=True
)

def to_dict(self):

    return {

        "id": self.id,

        "title": self.title,

        "destination": self.destination,

        "country": self.country,

        "description": self.description,

        "price": self.price,

        "duration": self.duration,

        "travel_type": self.travel_type,

        "image_url":
        f"https://images.unsplash.com{self.image_filename}",

        "is_available":
        self.is_available
    }
```

6) app/schemas.py

Validation des formats entrants.

</> Python



▶ Exécuter

```
from marshmallow import Schema, fields
```

```
class TripSchema(Schema):
```

```
    title = fields.Str(  
        required=True  
    )
```

```
    destination = fields.Str(  
        required=True  
    )
```

```
    country = fields.Str(  
        required=True  
    )
```

```
    description = fields.Str(  
        required=True  
    )
```

```
    price = fields.Float(  
        required=True  
    )
```

```
    duration = fields.Integer(  
        required=True  
    )
```

```
    travel_type = fields.Str(  
        required=True  
    )
```

```
    image = fields.Str(  
        required=True  
    )
```

```
    is_available = fields.Bool()
```

7) app/validators.py

Règles métier.

</> Python



▶ Exécuter

```
from urllib.parse import urlparse

ALLOWED_TYPES = [
    "city",
    "nature",
    "beach",
    "adventure"
]

def validate_trip(data):

    if data["price"] <= 0:

        return "Le prix doit être positif"

    if data["duration"] <= 0:

        return "La durée doit être positive"

    if data["travel_type"] not in ALLOWED_TYPES:

        return "Type de voyage invalide"

    image = urlparse(
        data["image"]
    )

    if (
        "images.unsplash.com"
        not in image.netloc
    ):

        return "Image Unsplash obligatoire"

    return None
```

8) app/utils.py

Fonctions utilitaires.

</> Python



▶ Exécuter

```
from urllib.parse import urlparse

def extract_image_path(url):

    return urlparse(
        url
    ).path

def build_image_url(path):

    return (
        "https://images.unsplash.com"
        + path
    )
```

9) app/services/__init__.py

Vide.

10) app/services/trip_service.py

Logique métier.

</> Python



▶ Exécuter

```
from ..extensions import db
from ..models import Trip

from ..utils import extract_image_path

def get_all_trips():

    return db.session.execute(
        db.select(Trip)
```



```
    ).scalars().all()

def get_trip_by_id(id):

    return db.session.get(
        Trip,
        id
    )

def create_trip(data):

    trip = Trip(

        title=data["title"],

        destination=data["destination"],

        country=data["country"].upper(),

        description=data["description"],

        price=float(data["price"]),

        duration=int(data["duration"]),

        travel_type=data["travel_type"],

        image_filename=
        extract_image_path(
            data["image"]
        ),

        is_available=
        data.get(
            "is_available",
            True
        )
    )

    db.session.add(
        trip
    )

    db.session.commit()
```

```
        return trip

def delete_trip(trip):

    db.session.delete(
        trip
    )

    db.session.commit()
```

11) app/routes/__init__.py

Vide.

12) app/routes/trips.py

Routes API.

 Python



 Exécuter

```
from flask import (
    Blueprint,
    jsonify,
    request
)

from ..schemas import TripSchema

from ..validators import validate_trip

from ..services.trip_service import (
    get_all_trips,
    get_trip_by_id,
    create_trip,
    delete_trip
)

trips_bp = Blueprint(
    "trips",
    __name__
)
```

```
schema = TripSchema()

@trips_bp.route(
    "/trips",
    methods=["GET"]
)
def list_trips():

    trips = get_all_trips()

    return jsonify(
        [
            t.to_dict()
            for t in trips
        ]
    )

@trips_bp.route(
    "/trips/<int:id>",
    methods=["GET"]
)
def get_trip(id):

    trip = get_trip_by_id(
        id
    )

    if not trip:

        return jsonify(
            {
                "error":
                    "Trip not found"
            }
        ), 404

    return jsonify(
        trip.to_dict()
    )
```

```
@trips_bp.route(
    "/trips",
    methods=["POST"]
)
def add_trip():

    body = request.json

    error = validate_trip(
        body
    )

    if error:

        return jsonify(
            {
                "error":error
            }
        ),400

    trip = create_trip(
        body
    )

    return jsonify(
        trip.to_dict()
    ),201


@trips_bp.route(
    "/trips/<int:id>",
    methods=["DELETE"]
)
def remove_trip(id):

    trip = get_trip_by_id(
        id
    )

    if not trip:
```

```
        return jsonify(  
            {  
                "error":  
                "Trip not found"  
            }  
        ), 404  
  
    delete_trip(  
        trip  
    )  
  
    return "", 204
```

Installation des dépendances

</> Bash



```
pip install flask  
pip install flask-cors  
pip install flask-sqlalchemy  
pip install marshmallow
```

Lancement

Depuis travel_api/ :

</> Bash



```
python run.py
```

API disponible :

```
http://127.0.0.1:5000
```



Routes :

```
GET    /trips  
GET    /trips/1  
POST   /trips  
DELETE /trips/1
```



Cette version est maintenant organisée comme une **vraie API Flask professionnelle** avec séparation :

Routes



Services



Models



Database



C'est une base solide pour brancher ton frontend React.

